

Quickstart

Note

Before proceeding, install Manim and make sure it is running properly by following the steps in [Installation](#). For information on using Manim with Jupyterlab or Jupyter notebook, go to the documentation for the [IPython magic command](#), `%%manim`.

Important

If you installed Manim in the recommended way, using the Python management tool `uv`, then you either need to make sure the corresponding virtual environment is activated (follow the instructions printed on running `uv venv`), or you need to remember to prefix the `manim` command in the console with `uv run`; that is, `uv run manim ....`

Overview

This quickstart guide will lead you through creating a sample project using Manim: an animation engine for precise programmatic animations.

First, you will use a command line interface to create a `Scene`, the class through which Manim generates videos. In the `Scene` you will animate a circle. Then you will add another `Scene` showing a square transforming into a circle. This will be your introduction to Manim's animation ability. Afterwards, you will position multiple mathematical objects (`Mobject`s). Finally, you will learn the `.animate` syntax, a powerful feature that animates the methods you use to modify `Mobject`s.

Starting a new project

Start by creating a new folder:

```
manim init project my-project --default
```

The `my-project` folder is the root folder for your project. It contains all the files that Manim needs to function, as well as any output that your project produces.

[Skip to content](#)

 [en](#)  [stable](#) ▼

Animating a circle

1. Open a text editor, such as Notepad. Open the file `main.py` in the `my-project` folder. It should look something like this:

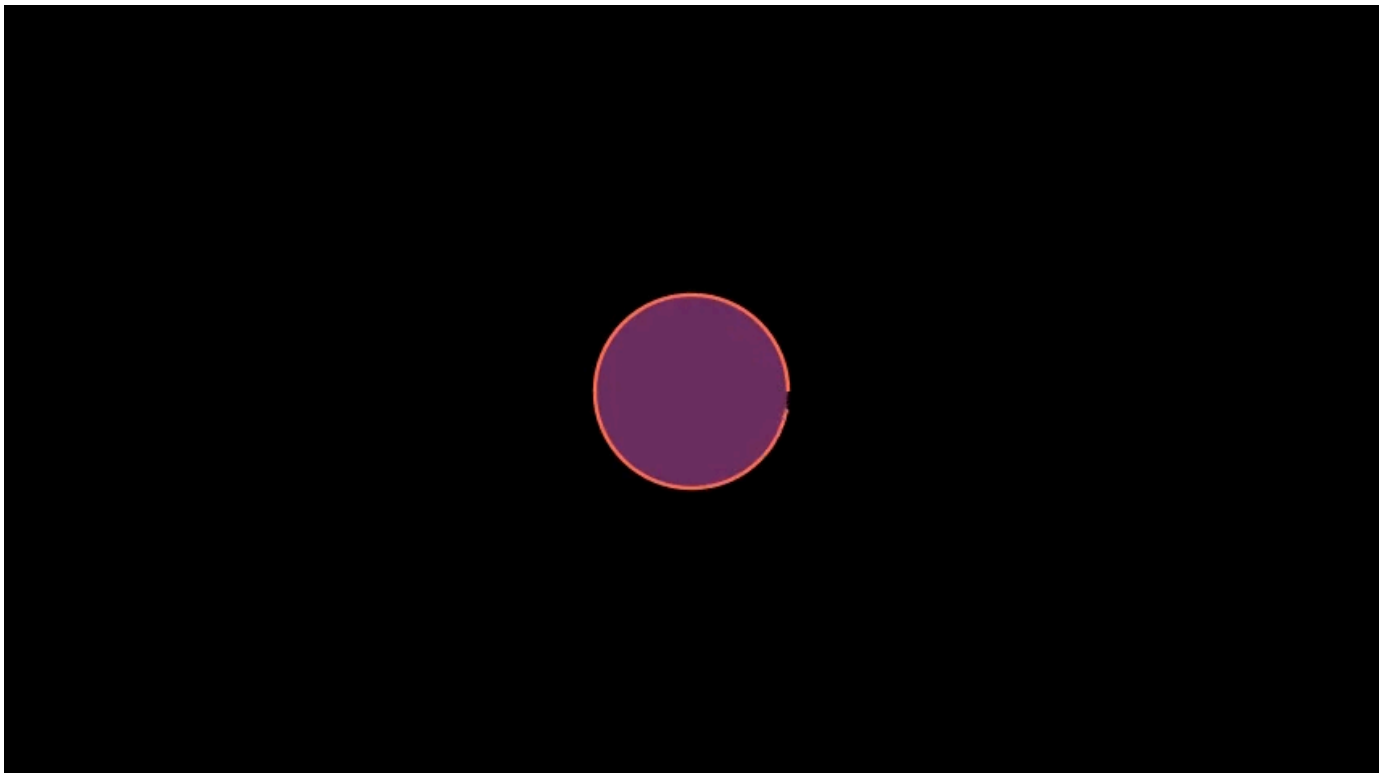
```
from manim import *

class CreateCircle(Scene):
    def construct(self):
        circle = Circle() # create a circle
        circle.set_fill(PINK, opacity=0.5) # set the color and transparency
        self.play(Create(circle)) # show the circle on screen
```

2. Open the command line, navigate to your project folder, and execute the following command:

```
manim -pql main.py CreateCircle
```

Manim will output rendering information, then create an MP4 file. Your default movie player will play the MP4 file, displaying the following animation.



If you see an animation of a pink circle being drawn, congratulations! You just wrote your first Manim scene from scratch.

Skip to content [message](#) instead, you do not see a video, or if  **en**  **stable** ▼

look like the preceding animation, it is likely that Manim has not been installed correctly. Please refer to our [FAQ section](#) for help with the most common issues.

Explanation

Let's go over the script you just executed line by line to see how Manim was able to draw the circle.

The first line imports all of the contents of the library:

```
from manim import *
```

This is the recommended way of using Manim, as a single script often uses multiple names from the Manim namespace. In your script, you imported and used `Scene`, `Circle`, `PINK` and `Create`.

Now let's look at the next two lines:

```
class CreateCircle(Scene):  
    def construct(self):  
        [...]
```

Most of the time, the code for scripting an animation is entirely contained within the `construct()` method of a `Scene` class. Inside `construct()`, you can create objects, display them on screen, and animate them.

The next two lines create a circle and set its color and opacity:

```
circle = Circle() # create a circle  
circle.set_fill(PINK, opacity=0.5) # set the color and transparency
```

Finally, the last line uses the animation `Create` to display the circle on your screen:

```
self.play(Create(circle)) # show the circle on screen
```

Tip

All animations must reside within the `construct()` method of a class derived from `Scene`. Other code, such as auxiliary or mathematical functions, may reside outside the class.

Transforming a square into a circle

With our circle animation complete, let's move on to something a little more complicated.

1. Open `scene.py`, and add the following code snippet below the `CreateCircle` class:

[Skip to content](#)

  en  stable ▼

```

class SquareToCircle(Scene):
    def construct(self):
        circle = Circle() # create a circle
        circle.set_fill(PINK, opacity=0.5) # set color and transparency

        square = Square() # create a square
        square.rotate(PI / 4) # rotate a certain amount

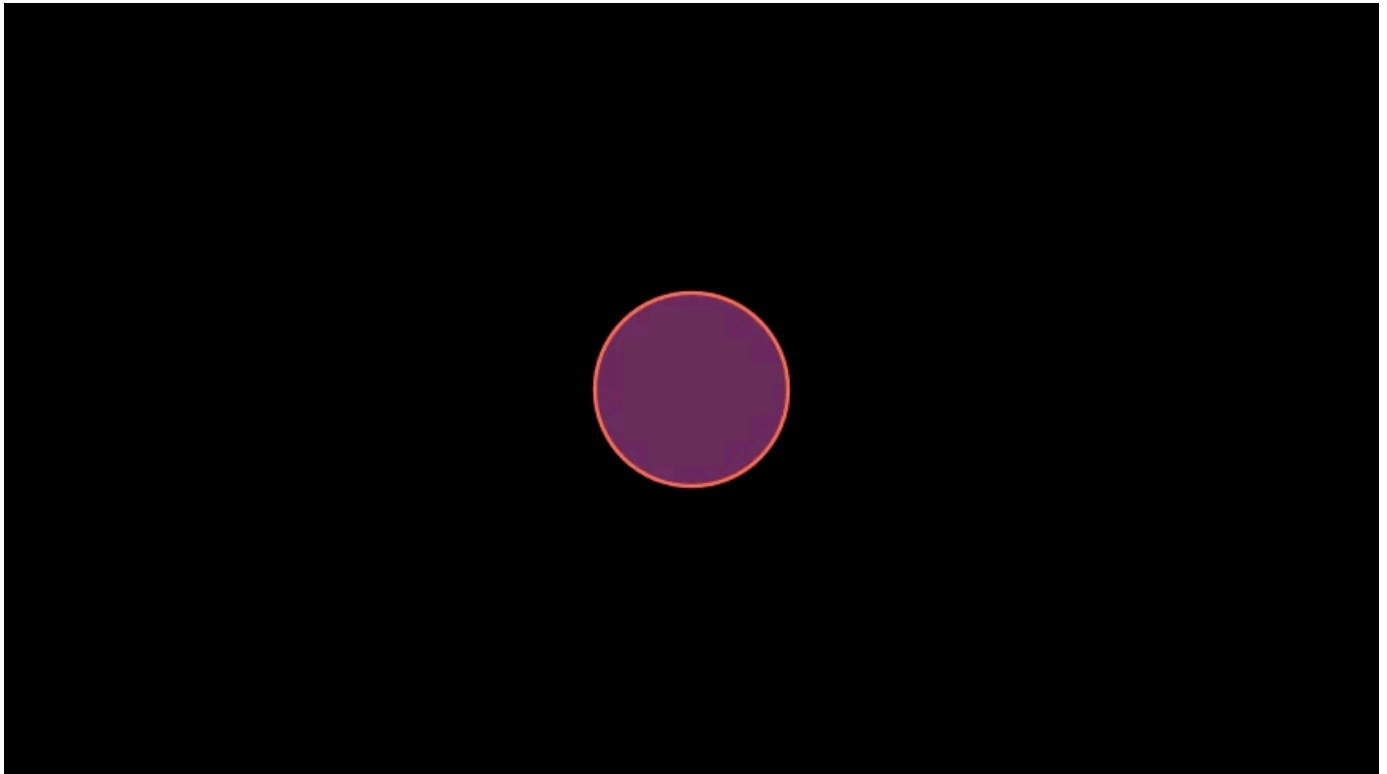
        self.play(Create(square)) # animate the creation of the square
        self.play(Transform(square, circle)) # interpolate the square into the circle
        self.play(FadeOut(square)) # fade out animation

```

2. Render `SquareToCircle` by running the following command in the command line:

```
manim -pql scene.py SquareToCircle
```

The following animation will render:



This example shows one of the primary features of Manim: the ability to implement complicated and mathematically intensive animations (such as cleanly interpolating between two geometric shapes) with just a few lines of code.

Positioning Objects

some basic techniques for positioning Objects

Skip to content

en stable

1. Open `scene.py`, and add the following code snippet below the `SquareToCircle` method:

```

class SquareAndCircle(Scene):
    def construct(self):
        circle = Circle() # create a circle
        circle.set_fill(PINK, opacity=0.5) # set the color and transparency

        square = Square() # create a square
        square.set_fill(BLUE, opacity=0.5) # set the color and transparency

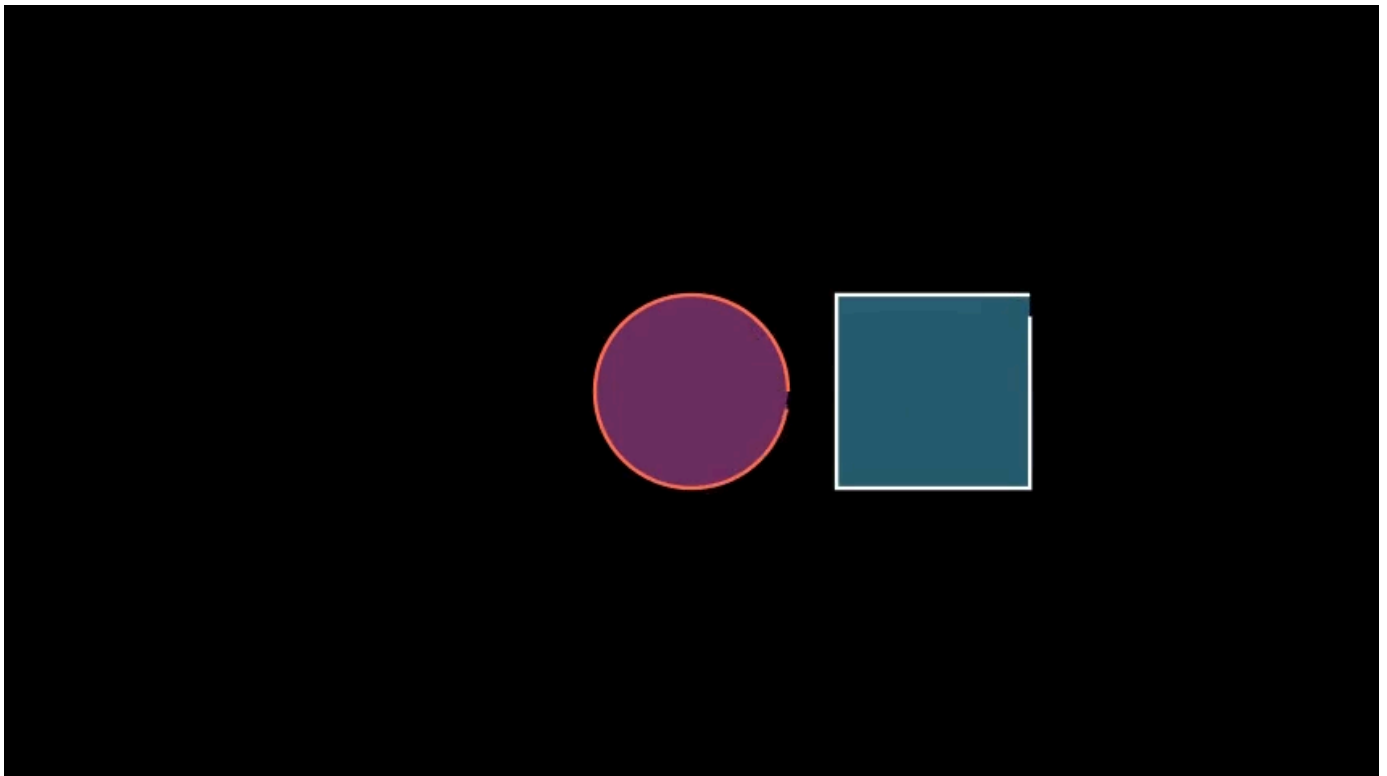
        square.next_to(circle, RIGHT, buff=0.5) # set the position
        self.play(Create(circle), Create(square)) # show the shapes on screen

```

2. Render `SquareAndCircle` by running the following command in the command line:

```
manim -pql scene.py SquareAndCircle
```

The following animation will render:



`next_to` is a `Mobject` method for positioning `Mobject`s.

We first specified the pink circle as the square's reference point by passing `circle` as the method's first argument. The second argument is used to specify the direction the `Mobject` is placed relative to the reference point. In this case, we set the direction to `RIGHT`, telling Manim to position the square to the right of the circle. Finally, `buff=0.5` applied a small distance buffer between the two objects.

Skip to content [IT to LEFT, UP, or DOWN instead, and see how t](#)



en



stable



Using positioning methods, you can render a scene with multiple `Mobject`s, setting their locations in the scene using coordinates or positioning them relative to each other.

For more information on `next_to` and other positioning methods, check out the list of `Mobject` methods in our reference manual.

Using `.animate` syntax to animate methods

The final lesson in this tutorial is using `.animate`, a `Mobject` method which animates changes you make to a `Mobject`. When you prepend `.animate` to any method call that modifies a `Mobject`, the method becomes an animation which can be played using `self.play`. Let's return to `SquareToCircle` to see the differences between using methods when creating a `Mobject`, and animating those method calls with `.animate`.

1. Open `scene.py`, and add the following code snippet below the `SquareAndCircle` class:

```
class AnimatedSquareToCircle(Scene):
    def construct(self):
        circle = Circle() # create a circle
        square = Square() # create a square

        self.play(Create(square)) # show the square on screen
        self.play(square.animate.rotate(PI / 4)) # rotate the square
        self.play(Transform(square, circle)) # transform the square into a circle
        self.play(
            square.animate.set_fill(PINK, opacity=0.5)
        ) # color the circle on screen
```

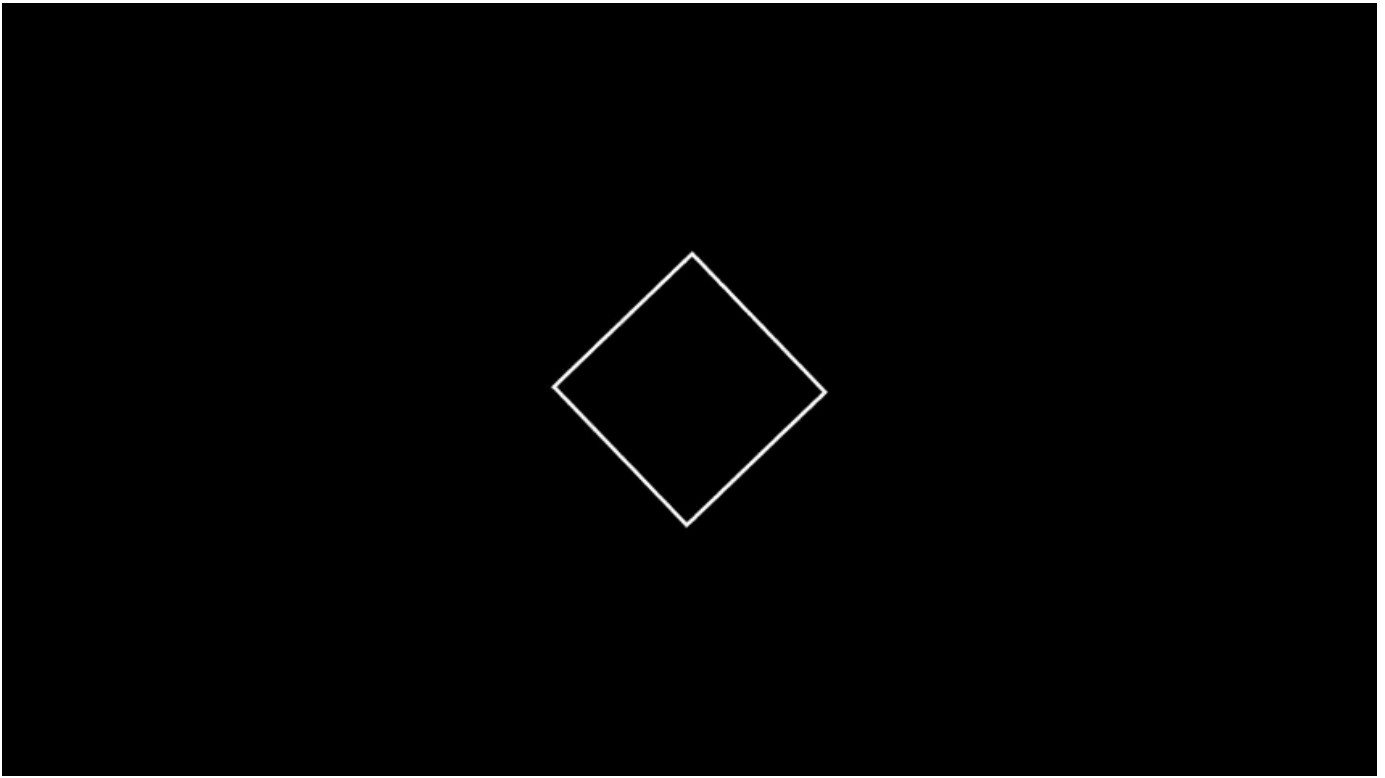
2. Render `AnimatedSquareToCircle` by running the following command in the command line:

```
manim -pql scene.py AnimatedSquareToCircle
```

The following animation will render:

[Skip to content](#)

  [en](#)  [stable](#) ▼



The first `self.play` creates the square. The second animates rotating it 45 degrees. The third transforms the square into a circle, and the last colors the circle pink. Although the end result is the same as that of `SquareToCircle`, `.animate` shows `rotate` and `set_fill` being applied to the `Mobject` dynamically, instead of creating them with the changes already applied.

Try other methods, like `flip` or `shift`, and see what happens.

3. Open `scene.py`, and add the following code snippet below the `AnimatedSquareToCircle` class:

```
class DifferentRotations(Scene):
    def construct(self):
        left_square = Square(color=BLUE, fill_opacity=0.7).shift(2 * LEFT)
        right_square = Square(color=GREEN, fill_opacity=0.7).shift(2 * RIGHT)
        self.play(
            left_square.animate.rotate(PI), Rotate(right_square, angle=PI), run_time=2
        )
        self.wait()
```

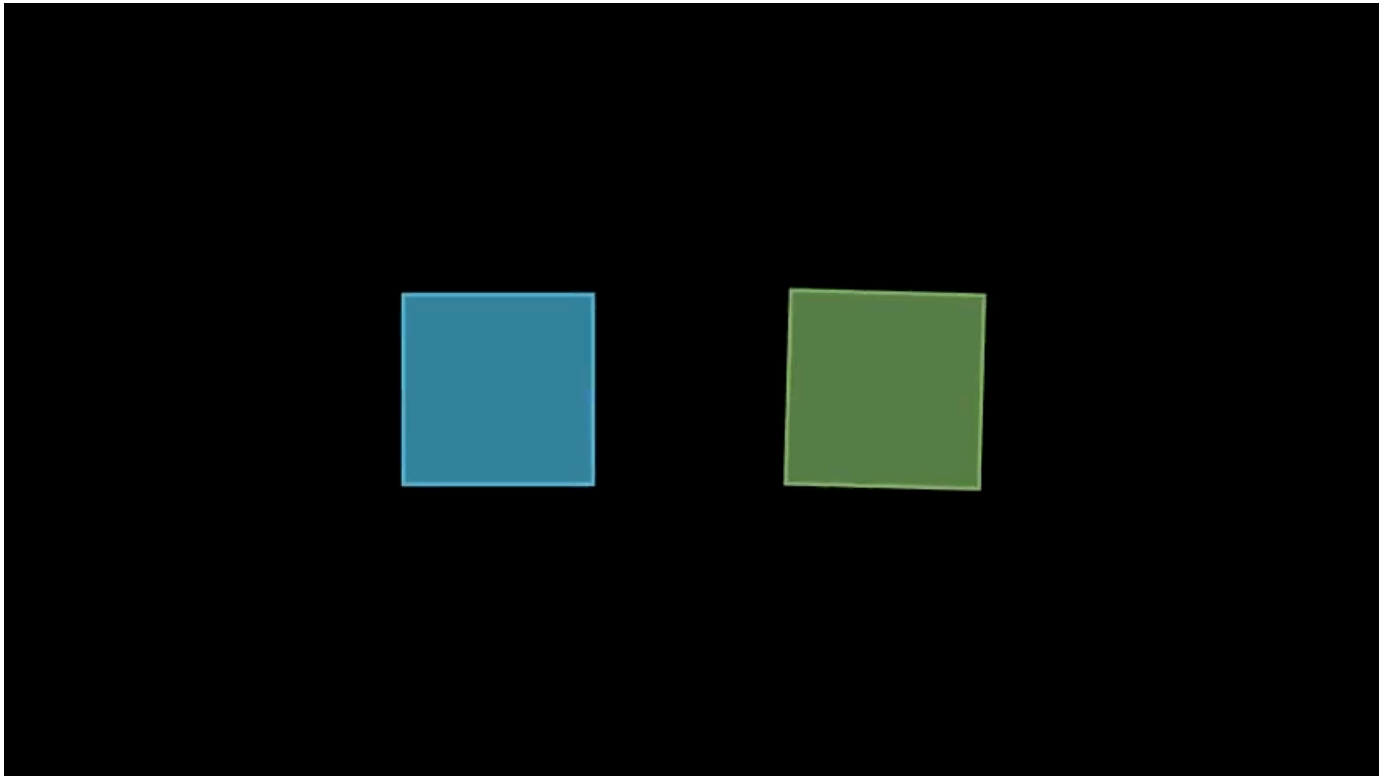
4. Render `DifferentRotations` by running the following command in the command line:

```
manim -pql scene.py DifferentRotations
```

The following animation will render:

Skip to content

  en  stable ▼



This scene illustrates the quirks of `.animate`. When using `.animate`, Manim actually takes a `Mobject`'s starting state and its ending state and interpolates the two. In the `AnimatedSquareToCircle` class, you can observe this when the square rotates: the corners of the square appear to contract slightly as they move into the positions required for the first square to transform into the second one.

In `DifferentRotations`, the difference between `.animate`'s interpretation of rotation and the `Rotate` method is far more apparent. The starting and ending states of a `Mobject` rotated 180 degrees are the same, so `.animate` tries to interpolate two identical objects and the result is the left square. If you find that your own usage of `.animate` is causing similar unwanted behavior, consider using conventional animation methods like the right square, which uses `Rotate`.

Transform VS ReplacementTransform

The difference between `Transform` and `ReplacementTransform` is that `Transform(mob1, mob2)` transforms the points (as well as other attributes like color) of `mob1` into the points/attributes of `mob2`.

`ReplacementTransform(mob1, mob2)` on the other hand literally replaces `mob1` on the scene with `mob2`.

The use of `ReplacementTransform` or `Transform` is mostly up to personal preference. They can be used to accomplish the same effect, as shown below.

[Skip to content](#)

 [en](#)  [stable](#) ▼

```
class TwoTransforms(Scene):
    def transform(self):
        a = Circle()
        b = Square()
        c = Triangle()
        self.play(Transform(a, b))
        self.play(Transform(a, c))
        self.play(FadeOut(a))

    def replacement_transform(self):
        a = Circle()
        b = Square()
        c = Triangle()
        self.play(ReplacementTransform(a, b))
        self.play(ReplacementTransform(b, c))
        self.play(FadeOut(c))

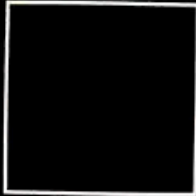
    def construct(self):
        self.transform()
        self.wait(0.5) # wait for 0.5 seconds
        self.replacement_transform()
```

However, in some cases it is more beneficial to use `Transform`, like when you are transforming several mobjects one after the other. The code below avoids having to keep a reference to the last mobject that was transformed.

[Skip to content](#)

  en  stable ▼

Example: TransformCycle



```
from manim import *

class TransformCycle(Scene):
    def construct(self):
        a = Circle()
        t1 = Square()
        t2 = Triangle()
        self.add(a)
        self.wait()
        for t in [t1, t2]:
            self.play(Transform(a, t))
```

Make Interactive

You're done!

With a working installation of Manim and this sample project under your belt, you're ready to start creating animations of your own. To learn more about what Manim is doing under the hood, move on to the next tutorial: [Manim's Output Settings](#). For an overview of Manim's features, as well as its configuration and other settings, check out the other [Tutorials](#). For a list of all available features, refer to the [Reference Manual](#) page.